Язык программирования C++

Введение в объектно-ориентированное программирование

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Объектно-ориентированное программирование

ООП – это один из принципов организации кода (парадигма).

Программа состоит из набора взаимодействующих объектов.

Объект – это сущность, принадлежащая определённому типу данных (классу) и объединяющая другие данные (поля) и поведение (методы).

Основные принципы:

- Инкапсуляция;
- Наследование;
- Полиморфизм.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Класс

Класс – это пользовательский тип данных, в котором объединены переменные других типов данных и функции, обрабатывающие эти данные и/или выполняющие операции по взаимодействию с окружающей средой.

При этом происходит сокрытие данных, т.е. получить доступ к ним извне класса можно только через функции этого класса.

Данные, объединённые в классе, также называют **полями** класса.

Функции, реализованные внутри класса, также называют **методами** класса.

Совокупность методов называется **интерфейсом** класса.

Поля и методы называют **элементами** класса.

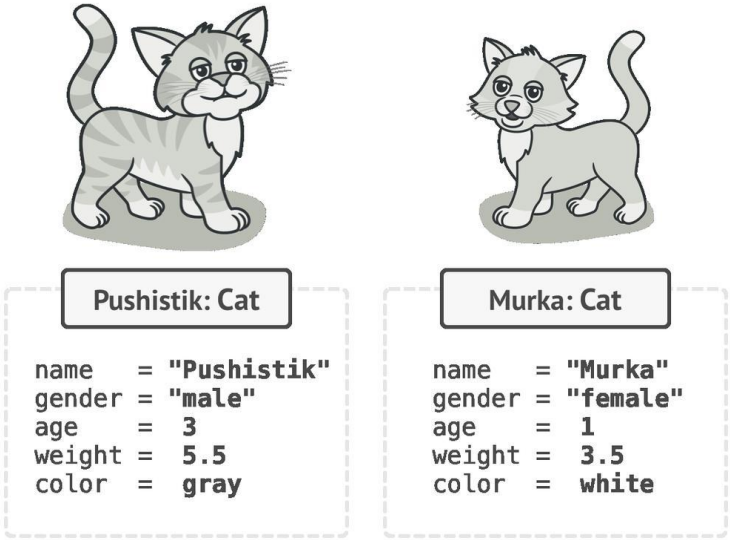
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объект

Объект класса – это представитель класса, построенный по хранящемуся в классе описанию.

Или иначе:

Объект класса – это **переменная**, типом которой является заданный класс.



Работа с объектом происходит только через специально предназначенные для этого методы.

Доступ к данным объекта, минуя его методы, запрещён!

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Инкапсуляция

Основные идеи инкапсуляции:

- **Объединение данных и методов:**
  - Поля и методы объединяются внутри класса.
  - Например, класс Car может содержать поле speed и методы accelerate(), brake().
- **Сокрытие реализации:**
  - Внутренние данные объекта (поля) часто объявляются как private или protected, чтобы запретить прямой доступ извне.
  - Для работы с этими данными предоставляются public-методы (геттеры и сеттеры).

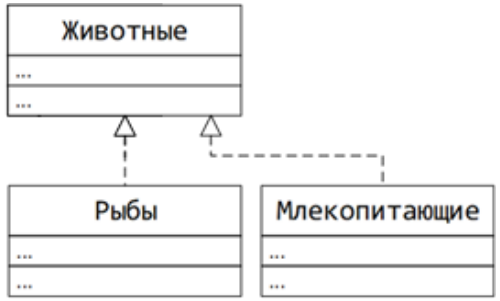
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Наследование

Наследование – это механизм, при котором класс-потомок перенимает все свойства и методы класса-родителя, а также может добавлять свои собственные или изменять унаследованные.

Основные понятия:

- **Базовый класс** (родительский, суперкласс) — класс, от которого наследуются свойства и методы.
- **Производный класс** (наследник, подкласс) — класс, который наследует от базового и может расширять или изменять его функциональность.



Для чего нужно:

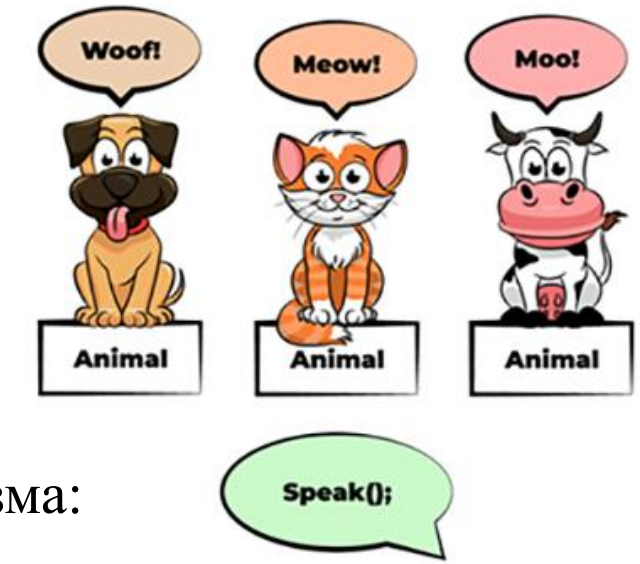
- Повторное использование кода (избежание дублирования).
- Создание иерархии классов (логическая организация кода).
- Расширяемость — можно добавлять новую функциональность в наследниках, не меняя базовый класс.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Полиморфизм

Полиморфизм – возможность использования одного и того же имени метода для работы с различными объектами базового и производных от него классов.

Концепция: «**один интерфейс – много методов**».



Различаются следующие виды полиморфизма:

- **Статический** – на этапе компиляции через перегрузку функций.
- **Динамический** – во время выполнения программы, реализуется с помощью виртуальных функций.
- **Параметрический** – на этапе компиляции, с использованием механизма шаблонов.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис объявления класса

Объявление класса можно выполнить следующим образом:

```
class Имя_класса
{
private: // используется по умолчанию
    закрытые элементы класса (поля и методы)
public:
    открытые элементы класса
protected:
    защищенные элементы класса
}; // Обратите внимание на точку с запятой!
```

Имя класса – произвольно выбираемый идентификатор;

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

В теле класса могут быть

Поля:

- **Константы** – хранят неизменяемые значения, связанные с классом.
- **Переменные** – содержат данные класса.

Методы:

- **Методы** – функции, которые реализуют вычисления или другие действия, выполняемые классом или объектом.
- **Конструкторы и деструкторы** – специальные методы, реализующие действия по инициализации объекта класса и действия по его уничтожению.
- **Операции** – задают действия с объектами с помощью знаков операций (+, - и т.д.).

Типы:

- **Типы** – это пользовательские типы данных (структуры, перечисления, другие классы), внутренние по отношению к классу (т.е. объявленные внутри этого класса).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример класса

```
class Box { // Класс является моделью коробки
private: // В большинстве случаев поля класса размещают
         // под спецификаторами private и protected.
    double ln; // length – длина
    double w; // width – ширина
public: // Для примера объявим поле he открытым
    double h; // height – высота
public: // В большинстве случаев методы класса размещают
        // под спецификатором public.
    // Метод возвращает объём коробки (методу доступны все
    double volume() const // поля класса)
    { return  ln * w * h; }
    // В простых случаях для получения значений полей исполь-
    double get_ln() const { return ln; } // зуются такие мет.
    // а для установки значений подобные:
    void set_ln(double _ln) { ln = _ln < 0 ? 0 : _ln; }
    // У этого метода реализация вне класса (в классе только
    double square() const; // его заголовок)
   // Другие методы
};
```

Означает, что метод не будет менять значения полей.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление объекта

Объект класса – это переменная типа класса.

Объявляется аналогично обычной переменной встроенного типа:

Имя_класса имя_объекта;

Создание объекта, указателя на объект и ссылки для класса **Box**:

```
Box d; // создаётся объект класса Box
Box * dptr = &d; // создаётся указатель на объект d
Box & dref = d; // создаётся ссылка на объект d
```

Для доступа к открытым полям (**public**) используются конструкции:

имя_объекта.имя_поля **указ_на_объект->имя_поля**

Например, можно работать напрямую с полем **h** объекта **d**:

```
cout << d.h; // выведет на экран текущее значение
dptr->h = 1.34; // присвоит новое значение 1.34
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Реализация метода внутри класса

Методам доступны все поля класса, их **передавать не нужно**!

Реализовать метод можно внутри или вне класса.

Внутри класса:

```
class Имя_класса
{
public:
  тип_возвр_зн  название_метода (список_параметров)
  {
    // реализация метода
  }
};
```

```
class Box {
…
public:
  double volume() {
    return  ln * w * h;
  }
  …
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис реализации метода вне класса

Вне класса:

```
class Имя_класса
{
public:
  тип_возвр_зн название_метода (список_параметров);
}; // Класс закрыт

тип_возвр_зн  Имя_класса::название_метода
                                (список_параметров)
{
  // реализация метода
}
```

```
class Box {
…
public:
  double square() const;
  …
};

double Box::square() const {
  return ln * w;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис вызова метода

Синтаксис вызова метода:

имя_объекта.имя_метода( ) **указ_на_объект->имя_метода()**

Например, вызвать метод **volume**( ) для объекта **d** можно так:

```
double vol = d.volume();
cout << "Объём коробки d = " << vol << endl; // или так:
cout << "Объём коробки d = " << d.volume() << endl;
```

Если поле объявлено как **private** или **protected**, то для доступа к данным необходимо написать отдельные методы следующего вида:

имя_объекта.get_имя_поля()

имя_объекта.set_имя_поля()

Например, можно работать с полем **ln** объекта **d**:

```
d.set_ln(3.3); // присвоит новое значение 3.3
cout << d.get_ln(); // выведет текущее значение поля
```

Разница с прямым доступом – можно добавить проверку на неотрицательность значений при присваивании.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример методов для ввода/вывода

```
class Box {
  int ln, w, h;
public:
  void print(ostream &out) const{
    out << "Коробка размером "
        << ln << " x " << w
        << " x " << h << endl;
  }
  void scan(istream &in) {
    if(&in == &cin) {
      cout << "Введите поля "
           << "коробки: ";
    }
    in >> ln >> w >> h;
  }
  // Другие
  // методы
};
```

```
Введите поля коробки: 2 1 5
Коробка размером 2 x 1 x 5
1: Введите поля коробки: 1 2 3
Коробка размером 1 x 2 x 3
2: Введите поля коробки: 5 4 8
Коробка размером 5 x 4 x 8
```

```
int main()
{
  Box b;
  b.scan(cin);
  b.print(cout);
  Box bs[2];
  for(int i = 0; i<2; i++)
  {
    cout << i+1 << ": ";
    bs[i].scan(cin);
    bs[i].print(cout);
  }
  return 0;
}
```

Создание массива объектов аналогично созданию массива структур.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Указатель this

Все **методы находятся в памяти в единственном экземпляре** и используются всеми объектами совместно.

Скрытый параметр **this** передаётся во все обычные методы и **хранит указатель на объект, вызвавший метод**.

Он никак не указывается, но считается, что этот параметр всегда стоит первым.

Может явно использоваться чтобы вернуть сам объект через return или явно выбрать поле при совпадении названия с параметром.

```
void  Box::set_ln(double  ln)
{
 this->ln = ln < 0 ? 0 : ln;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Конструкторы

Конструктор – это метод создания и инициализации объектов данного класса.

Уникальные свойства конструкторов :

- конструкторы имеют то же имя, что и класс;
- не возвращают значение и не имеют тип возвращаемого значения;
- конструктор вызывается неявно при создании или копировании объекта данного класса. Вызвать явно его нельзя.

Формат определения конструктора в теле класса может быть таким:

```
Имя_класса (список_параметров)
{
    // реализация конструктора
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Конструктор без параметров

Если конструктор без параметров в классе явно не определен, то будет использоваться **конструктор по умолчанию, созданный компилятором**.

Этот конструктор присваивает полям значения по умолчанию.

Если любой конструктор явно определен в классе, то конструктор по умолчанию не будет создаваться автоматически.

```
// Можно добавить в класс конструктор явно
Box()
{
   ln = w = h = 1;
}
```

```
Box a; // Здесь неявно вызывается
       // конструктор без параметров
cout << a.get_ln(); // Выведется 1
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Конструктор с параметрами

Класс может иметь **несколько конструкторов с разными параметрами** для разных видов инициализации.

Наличие в классе нескольких конструкторов называется **перегрузкой конструкторов**.

```
// определение конструктора находится внутри класса
Box(double _ln, double _w, double _h)
{
   ln = _ln; w = _w; h = _h;
}
```

```
Box a(3.4, 0.2, 1.0); // Здесь неявно вызывается
                      // конструктор с параметрами
cout << a.get_ln(); // Выведется 3.4
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Конструктор копирования

Конструктор копирования может принимать только один параметр – **константную ссылку** на другой объект типа X.

Если конструктор копирования в классе явно не определен, то он будет **создан компилятором автоматически**. Он копирует значения полей двух объектов **побайтово**.

Если полем является **указатель**, то будет **скопирован адрес**, а не значение. В этом случае необходимо реализовать конструктор копирования явно.

```
Box(const Box &t)
{
  ln = t.ln;
  w = t.w;
  h = t.h;
}
```

```
Box a(3.4, 0.2, 1.0);
Box z(a); // Вызов конструктора копир.
cout << z.get_ln(); // Выведется 3.4
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Конструктор перемещения

Конструктор перемещения может принимать только один параметр – **r-value ссылку** на **временный** объект типа X, который после работы этого конструктора будет сразу же удалён.

Конструкторы перемещения появились в стандарте C++11 и часто реализуются с использованием функции std::swap().

Основное назначение – снизить количество выделений и удалений динамической памяти при возврате локальных объектов из функций.

```
Box(Box && t)
{
   std::swap(ln, t.ln);
   std::swap(w, t.w);
   std::swap(h, t.h);
}
```

```
Box func() {
    Box res(3.4, 0.2, 1.0);
    return res;
}
// Создаём объект из возвращаемого
Box z(func());
cout << z.get_ln();
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Инициализация полей объектов

Два способа:

- передача значений параметров в тело конструктора;
- применение списка инициализаторов данных объекта. Этот список помещается между списком параметров и телом конструктора:

Имя_класса(список_параметров)

: список_инициализаторов_данных

{ тело конструктора }

Каждый инициализатор списка относится к элементу и имеет вид:

имя_элемента_данных (выражение)

Пример:

```
class Box {
   double ln, w, h;
   public:
   Box(double _ln, double _w)
      : ln(_ln), w(_w), h(0.5 * _ln) { }
};
```

Инициализация **Box A(5.0, 3.0)** присваивает элементам объекта **A** следующие значения: **A.ln=5.0, A.w=3.0, A.h=2.5**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Деструкторы

Деструктор – это метод уничтожения объектов данного класса.

Уникальные свойства деструктора:

- деструктор имеет то же имя, что и класс, но перед ним стоит символ ~ (тильда);
- не возвращает значение и не имеет тип возвращаемого значения;
- не принимает явных параметров;
- деструктор вызывается неявно при уничтожении объекта данного класса.

Формат определения деструктора в теле класса может быть таким:

```
~имя_класса()
{
    // реализация деструктора
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример деструктора

```
class Massiv
{
  int maxN ; // Макс.кол.элем.
  int curN; // текущее кол.элем.
  int *items; // массив
public:
  // Конструкторы
  Massiv() {
  maxN = curN = 0;
  items = NULL; }
  Massiv(int N, int C = 0) {
    maxN = N;
    curN = C;
    items = new int [maxN];
  }
  Massiv(const Massiv &other);
  // Деструктор
  ~Massiv( ) {
    if (maxN > 0)
      delete[] items;
  }
};
```

```
Massiv::Massiv(
        const Massiv &other) {
  if(items != 0)
    delete[] items;
  maxN = other.maxN;
  curN = other.curN;
  items = new int [maxN];
  for(int i = 0; i<maxN; i++)
    items[i] = other.items[i];
}
```

```
void main()
{
   Massiv *p = new Massiv[10];
   delete[] p;
   Massiv *a = new Massiv;
   delete a;
   Massiv b(80);
}
```

При уничтожении объектов (вызов оператора delete) деструктор вызывается автоматически.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Дружественная функция

Дружественная функция – это функция, не являющаяся методом класса, но имеющая доступ к его закрытым (*private*) и защищенным (*protected*) элементам.

Особенности дружественной функции:

- Объявляется внутри класса с модификатором **friend**.
- Не является элементом класса, поэтому ей **не передаётся** указатель **this**. Объект класса нужно передавать явно через параметр.
- **Не наследуется**.
- Объявляется внутри класса, а описана может быть как внутри класса, так и вне его.

```
class Foo {
  int data;
public:
  // описание внутри
  friend void Bar(Foo &f)
  {
    f.data = 1;
  }
};
```

Другом класса может быть:

- обычная функция: friend void f (...);
- метод другого класса: friend int Y::f (..);
- весь класс: friend class Y;

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций

Для перегрузки встроенной операции (+, -, != и т.д.) используется ключевое слово **operator**, после которого следует знак этой операции.

Перегружать операцию можно с помощью:

- метода класса;
- дружественной функции;
- обычной функции.

Перегружать нельзя следующие операции:

- **?:** (тернарный условный оператор);
- **::** (доступ к вложенным именам);
- **.** (доступ к полям);
- **sizeof** (нахождения размера в байтах);
- **typeid** (определение типа переменной).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Ограничения на перегрузку операций

- Нельзя определить новую операцию, например, operator**.
- Количество операндов, приоритет выполнения и ассоциативность операций (слева направо или справа налево) не изменяется.
- Как минимум один операнд должен быть пользовательского типа. Реализовать перегрузку операций для встроенных типов **нельзя**.
- Перегрузки операций наследуются (кроме операции =).
- Следующие операции можно перегрузить только **в качестве методов**:
  - **=** (присваивание);
  - **->** (доступ к элементам по указателю);
  - **( )** (обращение с объектом как с функцией);
  - **[ ]** (доступ по индексу к элементам какого-то поля);
  - операции приведения к типу;
  - операции управления памятью (**new**, **delete**).

Примечание. Перегружайте операторы только тогда, когда смысл перегрузки очевиден и не несёт скрытых сюрпризов! Например, операция + очевидна для матриц, но неочевидна для ящиков.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис перегрузки операций

Объявление функции-операции (т.е. её заголовок или прототип):

тип_возвр_зн operator знак_операции (список_параметров);

Сочетание **operator знак_операции** можно считать названием функции (метода). В этом случае, синтаксис объявления и определения перегрузки операции такой же, как у обычной функции (метода) языка C++.

Для определения количества параметров необходимо учитывать арность операции. Для перегрузки доступны унарные (например, ++) и бинарные (например, /).

n-арные операции перегружаются:

- методом с (n-1) параметром;
- функцией с n параметрами.

Перегрузка операций является, наряду с перегрузкой функций, одним из способов реализации принципа **полиморфизма**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка унарных операций

```
class Vec
{
 int x, y, z;
public:
 //конструктор по умолчанию
 Vec() : x(0), y(0), z(0) {}
 //прототип метода-операции для
 //перегрузки преф. инкремента
 Vec & operator ++();
 //прототип метода-операции для
 //перегрузки постф. инкремента
 Vec operator ++(int);
 //прототипы других методов
 //........
};
```

Здесь **int** – фиктивный параметр, указываемый для постфиксной операции, чтобы можно было отличить префиксную операцию от постфиксной. Он никак не указывается при вызове.

```
//Определение метода для
//перегрузки префиксного
//инкремента:
Vec & Vec::operator ++()
{
 x++; y++; z++;
 return *this;
}
//Определение метода для
//перегрузки постфиксного
//инкремента:
Vec Vec::operator ++(int)
{
 Vec tmp;
 tmp = *this;
 ++(*this);
 //Возвращает старое
 // значение объекта
 return tmp;
}
```

```
int main() {
 Vec v;
 Vec y = ++v;
 Vec z = v++;
 return 0;
}
//в v 2,2,2
//в y 1,1,1
//в z 1,1,1
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка бинарных операций

```
class Vec {
 double x, y, z;
public:
 // конструктор по умолчанию
 Vec() { x=0; y=0; z=0; }
 // конструктор, принимающий
 // параметры
 Vec(double x1, double y1,
     double z1)
 { x=x1; y=y1; z=z1; }
 // вывод полей
 void print() const {
  cout << "x=" << x << " y="
       << y << " z=" << z;
 }
 // метод для перегрузки +
 Vec operator+(const Vec &t) const;
 // метод для перегрузки -
 Vec operator-(const Vec &t) const;
};
```

```
// определение метода-операции +
Vec Vec::operator+(const Vec &t)
                          const
{
 Vec c;
 c.x = x + t.x;
 c.y = y + t.y;
 c.z = z + t.z;
 return c;
}

//определение метода-операции -
Vec Vec::operator-(const Vec &t)
                          const
{
 Vec c;
 c.x = x - t.x;
 c.y = y - t.y;
 c.z = z - t.z;
 return c;
}
```

```
int main() {
 Vec v(2,2,4);
 Vec z(3,1,0);
 Vec y = v+z;
 y.print();
 return 0;
}//x=5 y=3 z=4
```

Перегрузка операций сравнения

Операции сравнения сравнивают значения операндов, не изменяя их, и возвращают результат типа **bool**.

```
class Vec
{
 int x, y, z;
public:
 Vec(int x1, int y1, int z1)
  : x(x1), y(y1), z(z1) {}
 //прототип операции сравнения >
 bool operator >(const Vec &t)
                              const;
 //прототип операции сравнения <
 bool operator <(const Vec &t)
                              const;

 //...
};
```

```
int main() {
 Vec v(2,2,2);
 Vec z(1,1,1);
 if(v > z)
  cout << "v>z";
 return 0;
} //v>z
```

```
//Определение метода для
//перегрузки операции сравнения >
bool Vec::operator >(const Vec &t)
                              const
{
 int d1 = x*x + y*y + z*z;
 int d2 = t.x*t.x + t.y*t.y +
          t.z*t.z;
 return d1 > d2;
}

//Определение метода для
//перегрузки операции сравнения <
bool Vec::operator <(const Vec &t)
                              const
{
 return t > (*this);
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Преобразование типов

```
class Vec
{
  int x, y, z;
public:
  Vec(int x1, int y1, int z1) : x(x1), y(y1), z(z1) {}
  // метод-операция преобразования
  // к типу double
  operator double() const {
    return sqrt(double(x*x + y*y + z*z));
  }
  // метод-операция преобразования
  // к типу char*
  operator const char*() const
  {
    return "Class Vec";
  }
};
```

```
int main() {
 Vec z(0,3,4);
 double x = 2. +
        static_cast<double>(z);
 cout << x << endl; // 7
 cout << (const char*)z;
                    // Class Vec
 return 0;
}
```

Обратите внимание на отсутствие типа возвращаемого значения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование дружественных функций при перегрузке операций

```
class Vec
{
  double x, y, z;
public:
  // конструктор по умолчанию
  Vec() { x=0; y=0; z=0; }
  // конструктор, принимающий параметры
  Vec(double x1, double y1, double z1)
  { x=x1; y=y1; z=z1; }
  // вывод вектора
  void print() const;
  // функция-друг для перегрузки бинарной операции +
  friend Vec operator +(const Vec &op1, const Vec &op2);
  // функция-друг для перегрузки унарной операции ++
  friend Vec & operator ++(Vec &op1);
};
```

Обратите внимание на возвращение объекта для перегрузки сложения и ссылки на объект для перегрузки инкремента.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Использование дружественных функций при перегрузке операций

```
/* определение дружественной функции-операции для перегрузки
   операции + */
Vec operator +(const Vec &op1, const Vec &op2)
{
  Vec c;
  c.x = op1.x + op2.x;
  c.y = op1.y + op2.y;
  c.z = op1.z + op2.z;
  return c;
}
/* определение дружественной функции-оператора
   для перегрузки операции ++ */
Vec & operator ++(Vec &op1)
{
  op1.x++;
  op1.y++;
  op1.z++;
  return op1;
}
```

Возвращаем копию локального объекта

```
int main() {
 Vec v(2,2,2);
 Vec z(1,1,1);
 Vec res = v+z;
 ++res;
 res.print();
 return 0;
} //x=4 y=4 z=4
```

Возвращаем ссылку на уже существующий объект (не локальный)

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций >> и << для ввода–вывода встроенных типов

При вводе/выводе первым операндом является **поток**.

Класс istream определен как

```
class istream
{
// .....
public:
// символьная строка
istream & operator >>(char *);
// символ
istream & operator >>(char &);
istream & operator >>(short &);
istream & operator >>(int &);
istream & operator >>(long &);
istream & operator >>(float &);
istream & operator >>(double &);
//.....
};
```

Класс ostream определен как

```
class ostream
{
//.....
public:
ostream & operator <<(char *);
ostream & operator <<(int);
ostream & operator <<(long);
ostream & operator <<(double);
//.....
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций >> и << для ввода–вывода пользовательских типов

Поскольку на первом месте в параметрах обязательно должен быть поток, то перегрузку операций >> и << можно сделать **только функцией**.

```
class Vec
{
  int x, y, z;
public:
  // конструктор по умолчанию
  Vec() { x=0; y=0; z=0; }
  // конструктор, принимающий параметры
  Vec(int x1, int y1, int z1)
  { x=x1; y=y1; z=z1; }
  // вывод вектора
  ostream& print(ostream &stream);
  // функция-друг для вывода вектора
  friend ostream& operator <<(ostream &stream, const Vec &obj);
  // функция-друг для ввода вектора
  friend istream& operator >>(istream &stream, Vec &obj);
};
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций >> и << для ввода–вывода пользовательских типов

Или все необходимые действия прописать в методах, которые потом вызвать в обычных функциях.

```
// определение метода для вывода вектора в поток
ostream & print(ostream &stream) {
  stream << "x=" << x << ", ";
  stream << "y=" << y << ", ";
  stream << "z=" << z << "\n";
  return stream; // возврат потока
}
// определение функции для вывода вектора в поток
ostream & operator <<(ostream &stream, const Vec &obj) {
  return Vec.print(stream); // такой вариант предпочтительней, т.к.
}                           // не нарушает инкапсуляцию
// определение функции для ввода вектора из потока
istream & operator >>(istream &stream, Vec &obj)  {
  // аналогично с парой print - operator <<
}
```

```
int main() {
 ifstream in("in.txt");
 Vec z;
 in >> z; //читаем из файла
 cout << z;
 return 0;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева